tion. Here the binary representation of 1 to the base 10 is the same as 1 to the base 2 (for binary), and is moved left by 18 places which makes it 1000000000000000000 to the base 2 which is 262144 to the base 10. It is helpful to read up and educate yourself on the way binary language works so that you're fluent enough for more advanced projects. We only know these specific values for the LED project due to their documentation in the Raspberry Pi manual.

The storage location of the value in the registry are arrived at by understanding the settings of the GPIO. We know that the GPIO controller has a set of 24 bytes which control the pin. The first four of these bytes are connected to the first 10 GPIO pins, the next four bytes are connected to the next 10 pins and so on, totally 54 GPIO pins, which is six sets of 4 bytes. Each four byte section has three bits related to one of the ten GPIO pins. So in order to arrive at the 16th GPIO pin we need the second set of four bytes which controls pins 10 through 19. And since we need the sixth set of the three bits we arrive at the number 18 (which is 6 times 3) and insert it in the logical shift left instruction. The last line of instruction code therefore is "str" which stores the value from the first instruction in register r1 to the address calculated from the last instructions. This successfully executes the first of our two messages to the GPIO controller telling it to put the 16th pin on stand-by.

Final Activation

Now that the LED is on stand-by we need to give it the activation command. In the case of the Raspberry Pi the messaging system is slightly inverted, where the off instruction actually activates the LED. This means that as a matter of core design, the LED light turns on when the GPIO pin is turned off. Again, this is a matter of designer preference and follows no particular reason or pattern. The final lines of code are:

```
mov r1,#1
lsl r1,#16
str r1,[r0,#40]
```

Using all the familiar commands as before we execute the final instructions. Do not worry about understanding the values ascribed to the above commands - try and puzzle it out based on our discussion of GPIO design, binary language and hexadecimal numbering. The first line puts the number 1 in the r1 registry, the second line shifts the binary value representation of the number 1 in the r1 registry left by 16 places. Since we want the "off"